

恒致创造嵌入式软件开发规范 QRS-0002-2026

目 录				
1 目的				
2 适用范围				
3 总体要求				
4 命名与目录规范				
5 代码开发规范				
6 注释与文档规范				
7 版本控制规范				
8 版本发布规范				
9 参考资料				
修 订 历 史				
版本	修订内容	修订日期	修订者	批准者
V1.0	初版发布	2026-03-11	胡望伟	
V1.1	添加参考资料；简化分支管理	2026-04-02	胡望伟	

发文范围						受 控 印 章	
编制	胡望伟	审核		批准		收文 部门	
日期	2026-03-11	日期		日期			

1. 目的

建立统一、可执行的嵌入式软件开发规范，降低沟通成本，减少现场风险，提升代码可维护性、可追溯性和交付稳定性。

2. 适用范围

本规范适用于公司各类产品的嵌入式软件开发、维护与发布活动

3. 总体要求

3.1 执行原则

- 规范条款以少而硬为主，避免流于形式
- 能通过工具自动检查的，优先工具检查
- 能通过 Git 保留的信息，不重复手工维护
- 评审和发布流程应完整、清晰、可追溯

3.2 安全优先原则

涉及执行机构控制、远程控制、远程升级等功能时，必须优先考虑人员安全、设备安全和财产安全，至少覆盖以下问题：

- 异常情况下设备是否能进入预设安全状态，避免误动作、连续输出或失控运行
- 关键输出是否具备联锁、超时、限值和故障切断机制
- 异常、通信中断或升级中断时是否具备降级处理和告警机制
- 上电、复位、异常恢复和掉电恢复后的设备状态及关键参数是否一致、可控、可恢复

4. 命名与目录规范

4.1 文件命名

- 文件名必须全部小写，单词间使用下划线分隔
- 文件名应体现模块和功能，如 `drv_motor.c`、`bsp_uart.c`
- `.c` 与 `.h` 建议成对出现；头文件只放声明、宏、类型定义，不放变量定义和复杂实现

4.2 代码命名

类型	规则	示例
宏定义 / 常量	全大写 + 下划线	MAX_SPEED 、 TIMEOUT_MS
枚举值	建议带模块前缀	MOTOR_STATE_STOP
函数	小写 + 下划线, 建议使用 模块_动词_对象	motor_set_speed()
普通变量	小写 + 下划线	retry_count
全局变量	g_ 前缀	g_system_state
文件内静态变量	s_ 前缀	s_init_flag
指针变量	可使用 p_ 前缀	p_buffer
类型定义	小写 + _t 后缀	motor_state_t

4.3 目录建议

建议按职责分层组织目录：

app/	业务逻辑、状态机等
service/	公共服务，如参数、告警、存储、升级等
protocol/	通信协议
driver/	芯片外设驱动
bsp/	板级适配
common/	通用头文件、工具函数、错误码等
test/	单元测试或验证代码等
doc/	接口说明、发布说明等

4.4 禁止使用魔法数字

代码中不允许出现无意义的裸数值，必须使用宏、常量或枚举表达意图：

```
/* 禁止 */
if (speed > 3000U) {
    return -1;
}

/* 正确 */
#define MOTOR_MAX_RPM 3000U

if (speed > MOTOR_MAX_RPM) {
    return -1;
}
```

5. 代码开发规范

5.1 明确禁止事项

禁止行为	原因
在头文件中定义变量	容易导致重复定义与耦合扩散
变量未初始化直接使用	可能导致未定义行为
忽略返回值继续执行	错误被静默吞掉
共享变量无保护直接跨中断/任务访问	容易产生竞争问题
无超时地等待硬件或通信响应	容易导致死锁或系统卡死
在中断中执行阻塞或复杂逻辑	影响实时性和系统稳定性
运行期无约束使用动态内存	容易导致碎片、失败和难定位问题

5.2 格式要求

- 【必须】缩进使用 4 个空格，禁止使用 Tab
- 【必须】花括号不得省略，即使控制语句只有一行
- 【建议】每行不超过 120 个字符
- 【建议】统一使用 `.clang-format` 或 IDE 格式化配置

5.3 头文件要求

每个头文件必须有防重复包含保护：

```
#ifndef BSP_UART_H
#define BSP_UART_H

/* 内容 */

#endif /* BSP_UART_H */
```

头文件还应满足以下要求：

- 【必须】只暴露对外需要的接口
- 【必须】不在头文件中定义全局变量
- 【建议】不在头文件中包含无关头文件

5.4 函数与模块设计

- 【必须】模块对外接口稳定、清晰，禁止跨层直接访问其他模块的内部变量
- 【必须】模块初始化接口明确，如 `xxx_init()`、`xxx_start()`、`xxx_stop()`
- 【建议】单个函数应职责明确、规模适中；当函数代码过长或内部逻辑复杂时，应拆分为多个辅助函数
- 【建议】函数参数控制在 5 个以内，超过时优先考虑结构体封装

5.5 错误处理

- 【必须】不得忽略关键函数返回值
- 【必须】所有硬件、通信、存储、升级相关操作必须有明确错误返回
- 【必须】出现异常时必须明确处理方式：重试、降级、告警、复位或停止
- 【建议】错误码统一定义在公共头文件中

```
typedef enum
{
    SW_OK = 0,
    SW_ERR_PARAM = -1,
    SW_ERR_TIMEOUT = -2,
    SW_ERR_HW = -3
} sw_err_t;
```

5.6 超时与阻塞控制

- 【必须】所有等待硬件状态、通信响应、标志位变化的代码必须设置超时
- 【必须】需要重试时必须限定最大重试次数
- 【必须】周期任务、主循环和任务函数中，避免不可控阻塞
- 【必须】非必要场景下，禁止长时间忙等待

5.7 中断与并发规则

- 【必须】中断服务函数只做必要操作：置标志、搬运数据、通知任务
- 【必须】中断中禁止执行阻塞操作、复杂计算、动态内存分配、日志打印
- 【必须】中断与任务、任务与任务之间共享数据时，必须明确并发保护方式
- 【必须】简单标志位可用 `volatile`；涉及多步读写或结构体共享时，必须使用临界区、锁或消息机制

5.8 内存使用规则

- 【必须】所有缓冲区操作必须明确长度边界，禁止越界访问
- 【必须】指针在使用前必须确认有效性
- 【必须】长期运行的产品固件中，禁止运行期随意使用 `malloc/free`
- 【建议】如项目必须使用动态内存，说明用途、生命周期、失败处理和碎片风险

5.9 状态机与故障安全

- 【必须】对可能导致危险动作的输出，必须定义上电默认状态和故障安全状态
- 【必须】看门狗策略、异常复位策略、掉电恢复策略应在项目文档中说明
- 【建议】关键控制逻辑采用显式状态机实现，避免大量隐式标志位拼接
- 【建议】记录复位原因、关键故障码和最近一次异常上下文

5.10 编译与静态检查

- 【必须】编译器告警必须开启，新增代码不得引入新的 warning
- 【必须】发布版本必须能够完整编译通过

5.11 参数与配置管理

- 【必须】设备参数应与业务代码分离管理，关键参数须有范围校验
- 【必须】涉及安全保护、执行控制的关键参数须具备防误配措施，修改须有记录
- 【建议】参数项注明默认值、合法范围和单位

5.12 日志与诊断

- 【必须】系统具备基本日志或事件记录能力，用于故障定位和运行追溯
- 【必须】至少记录复位原因、故障码、升级结果和关键参数修改
- 【必须】日志内容可用于现场定位，不得仅输出无上下文的错误编号
- 【建议】存储空间受限的设备，明确日志保留数量、覆盖策略和导出方式

6. 注释与文档规范

6.1 文件头注释

文件头注释示例：

```
/**
 * @file      drv_motor.c
 * @brief     电机驱动模块
 * @author    张三
 * @date      2026-03-11
 */
```

- 文件修改历史以 Git 提交记录为准
- 对外发版说明、重大变更说明写入 `doc/` 或发布记录

6.2 函数注释

以下场景必须写函数注释：

- 对外公开接口
- 复杂算法或复杂状态转换
- 与硬件时序、寄存器、协议约束强相关的函数

示例：

```
/**
 * @brief 设置电机转速
 * @param speed_rpm 目标转速，单位 RPM
 * @retval SW_OK 设置成功
 * @retval SW_ERR_xx 设置失败
 * @note 调用前须完成电机初始化
 */
sw_err_t motor_set_speed(uint16_t speed_rpm);
```

6.3 行内注释

- 注释应解释“为什么”，而不是重复“做了什么”
- 对临界时序、硬件约束、规避缺陷的代码应补充说明
- 过时注释必须同步删除或更新

```
/* 不好 */
count++; /* count 加 1 */

/* 好 */
count++; /* 达到阈值后触发上报，避免单次抖动误判 */
```

6.4 最低文档要求

每个项目至少应维护以下文档或等价记录：

- 软件模块说明或目录说明
- 主要通信接口说明
- 关键参数说明
- 发布记录与本次验证结论
- 已知限制、风险点与注意事项

7. 版本控制规范

7.1 基本要求

- 所有工程必须纳入 Git 管理，提交到gitlab代码管理平台
- 密码、证书、生产密钥等敏感信息禁止提交
- 编译产物、日志、临时文件不得提交，必须写入 `.gitignore`
- 每次提交前应确保代码可编译，至少完成与本次改动相关的基本验证
- 涉及硬件驱动、时序控制、通信链路、升级、故障保护等重要改动，应优先在设备上完成验证；无设备时，在功能板上完成模拟验证

7.2 分支模型

分支模型定义如下：

dev	唯一长期主分支，承担日常开发、集成和发布基线管理
├─ feature/<功能名>	功能开发分支
├─ release/vX.Y.Z	发布准备分支（按需使用）
└─ hotfix/<问题描述>	紧急修复分支（按需使用）

规则如下：

- 小改动可直接在 `dev` 开发并提交；改动较大或风险较高时，从 `dev` 切出 `feature/*`
- `feature/*` 完成开发和必要验证后，合并回 `dev`
- 正式版本可直接基于 `dev` 打 Tag 发布；需要冻结测试时，从 `dev` 切出 `release/vX.Y.Z`
- `release/*` 仅允许进行发布准备、缺陷修复、版本信息更新和必要验证
- `release/*` 验证通过后，合并回 `dev` 并打 Tag： `vX.Y.Z`
- 紧急问题应从对应已发布版本的 Tag 或发布提交切出 `hotfix/*`
- `hotfix/*` 修复并验证通过后，合并回 `dev`，并根据需要补打修订版本 Tag
- 如存在正在验证中的 `release/*` 分支，且 hotfix 内容需纳入该版本，应同步合并到对应 `release/*`
- 临时分支合并后应删除，保持仓库整洁

7.3 Commit 规范

格式： <类型>: <简短描述>

类型	含义	示例
feat	新增功能或功能变更	feat: 增加设备联动控制逻辑
fix	问题修复	fix: 修复升级超时处理异常
docs	文档	docs: 更新接口说明
refactor	重构	refactor: 拆分串口通信模块
test	验证相关修改	test: 增加通信协议解析用例

类型	含义	示例
chore	构建、脚本或配置修改	chore: 更新编译选项

要求如下：

- 每次 commit 只做一件事
- 描述应具体，禁止使用 update 、 修改 、 fix bug 、 111 等无意义文本
- 提交前应清理调试代码、无效注释和临时文件

7.4 代码检查要求

- 代码提交前必须完成自查
- 涉及驱动、中断、通信协议、升级、参数存储、故障保护等高风险改动时，优先由其他成员评审或走查
- 如项目仅 1 人开发或当时无其他成员可评审，可由提交人自查后提交，但必须补充验证记录
- 高风险改动的自查或评审，至少覆盖变更范围、异常处理、影响接口、关键参数和回滚路径

8. 版本发布规范

8.1 版本号规则

格式：vX.Y.Z，字母 v 小写。

版本位	含义	触发条件
x	主版本	架构较大变化、不兼容变更、重大功能重构
y	次版本	新增功能、流程或接口扩展
z	修订版本	Bug 修复、小范围调整、兼容性修正

需要区分内部验证包和现场验证包时，可使用 _alpha.N 、 _beta.N 。

8.2 代码中的版本信息

版本信息应统一定义，不得在多处散写：

```
#define SW_VERSION_MAJOR 1
#define SW_VERSION_MINOR 0
#define SW_VERSION_PATCH 0
#define SW_VERSION_STR "v1.0.0"
#define BUILD_DATE __DATE__
#define BUILD_TIME __TIME__
```

建议增加以下可追溯信息：

- Git 短哈希
- 硬件版本号
- 关键配置版本

8.3 固件包命名

格式： <产品型号>_<版本号>_<Git短哈希>.bin

```
WashMachine_v1.2.3_a3f5c12.bin  
WashMachine_v1.2.3_beta.1_a3f5c12.bin
```

Git 短哈希命令：

```
git rev-parse --short HEAD
```

8.4 发布流程

发布流程如下：

1. 以当前发布基线完成开发收口，并通过与本次改动相关的最小验证
2. 根据项目需要，直接基于 `dev` 发布，或切出 `release/vX.Y.Z` 进行冻结测试
3. 在发布基线上完成版本号更新、必要验证、缺陷修复和发布确认
4. 发布确认后打 Tag: `vX.Y.Z`；如使用 `release/\*`，应先合并回 `dev`
5. 基于 Tag 构建正式固件并归档
6. 记录发布说明、验证结论、已知问题和版本对应关系

8.5 发布检查清单

每次发布前至少确认以下内容：

- 代码已合并到正确分支，Tag 与提交一致
- 版本号、构建时间、Git 哈希正确
- 完整编译通过，无新增 warning
- 已完成与本次改动相关的基本冒烟验证
- 目标硬件版本、配置文件、关键参数已确认
- 涉及驱动、中断、通信、存储、升级、故障保护等高风险改动时，已在设备上完成验证；无设备时，应完成功能板验证
- 涉及升级链路、参数迁移或回滚要求时，已验证升级流程、失败处理和回滚路径
- 固件包、发布说明、验证记录可追溯

8.6 发布时间管理

- 正式发布时间应避开业务高峰、节假日等高风险时段
- 发布前应明确发布时间窗口、发布负责人和回滚负责人；必要时明确发布后观察时长

8.7 升级与灰度发布规范

- 影响范围较大的远程升级应采用灰度发布，不得默认一次性全量升级
- 灰度发布应先小范围验证，再逐步扩大范围，确认通过后再全量发布
- 灰度期间如出现安全风险、关键功能异常、告警率明显升高或升级失败率异常，应立即停止扩大发布

8.8 升级回滚要求

- 版本发布前必须明确回滚路径、回滚条件和回滚步骤
- 升级失败后，设备应进入安全状态，并具备恢复、重试或人工处理路径
- 涉及 bootloader、分区切换、参数迁移或协议兼容性变更时，必须提前验证回滚可行性

9. 参考资料

下列外部公开资料供编写、培训与执行本规范时对照查阅。

主题	参考
Git 使用与分支	Git 官方文档
GitLab 合并请求、保护分支、CI	GitLab 文档
提交信息类型 (feat/fix 等)	Conventional Commits
语义化版本号 (MAJOR.MINOR.PATCH)	Semantic Versioning 2.0.0
注释与文档生成	Doxygen